DEVOPS WINTER ARC

90 Days Challenge • Kubernetes Week 3

DAY 19

Storage & StatefulSets

MISSION: Keep a database record alive—even after its Pod is deleted.

Focus	PV • PVC • StorageClass • StatefulSet • Headless Service
Time	3–4 hours | 30% concepts + 70% hands-on
Deliverable	A PostgreSQL StatefulSet that retains data after Pod recreation.

1. Scope rule — no repetition

Day 15 covered Kubernetes basics and a simple Nginx workload; Day 16 covered rollouts and rollback; Day 17 covered Service networking and DNS; Day 18 covered ConfigMaps, Secrets, and namespaces. **Do not repeat those tasks.** Day 19 is about durable storage, StatefulSets, PVC/PV binding, and stable identity for stateful apps.

2. Understand the storage chain

PV: storage resource available to the cluster. **PVC:** a workload's request for storage, including size and access mode. **StorageClass:** defines how storage can be provisioned. **StatefulSet:** manages Pods with stable identities and persistent storage patterns, such as **pg-0**. **Headless Service:** uses clusterIP: None and can provide DNS records for individual StatefulSet Pods. **Important:** a container's writable layer is not reliable durable storage; deleting a Pod does not normally delete its PVC.

3. Task A — inspect your cluster

Run and save the output. Do not assume your StorageClass name; verify it first.

```
kubectl get storageclass
kubectl get pv
kubectl get pvc -A
kubectl describe storageclass standard
```

If your default StorageClass is not named **standard**, update the manifest below to use the name your cluster reports.

4. Task B — create a lab namespace and dummy Secret

Use a dummy lab password only. Never commit real credentials or show password values in public screenshots.

```
kubectl create namespace day19-storage
kubectl create secret generic pg-secret \
--from-literal=POSTGRES_PASSWORD=WinterArcLab123! \
-n day19-storage
```

5. Task C — create the Headless Service

Save as **pg-headless.yaml**. The StatefulSet uses this Service for stable network identity.

```
apiVersion: v1
kind: Service
metadata:
name: pg-headless
namespace: day19-storage
spec:
clusterIP: None
selector:
app: pg
ports:
- name: postgres
port: 5432
targetPort: 5432
```

```
kubectl apply -f pg-headless.yaml
kubectl get svc -n day19-storage
```

6. Task D — create PostgreSQL StatefulSet

Save as **postgres-statefulset.yaml**. Notice the **volumeClaimTemplates** section: it creates a PVC for the StatefulSet Pod. The database stores its files under the mounted data directory.

```
apiVersion: apps/v1
kind: StatefulSet
metadata:
```

```
name: pg
namespace: day19-storage
spec:
serviceName: pg-headless
replicas: 1
selector:
matchLabels:
app: pg
template:
metadata:
labels:
app: pg
spec:
containers:
- name: postgres
image: postgres:16-alpine
ports:
- containerPort: 5432
env:
- name: POSTGRES_DB
value: winterarc
- name: POSTGRES_USER
value: appuser
- name: POSTGRES_PASSWORD
valueFrom:
secretKeyRef:
name: pg-secret
key: POSTGRES_PASSWORD
volumeMounts:
- name: data
mountPath: /var/lib/postgresql/data
volumeClaimTemplates:
- metadata:
name: data
spec:
accessModes: ["ReadWriteOnce"]
storageClassName: standard
resources:
requests:
storage: 1Gi
```

```
kubectl apply -f postgres-statefulset.yaml
kubectl get statefulset,pods,pvc -n day19-storage -w
```

7. Task E — create a record and test persistence

Wait for **pg-0** to become Ready and its PVC to become Bound. Open the database shell:

```
kubectl exec -it -n day19-storage pg-0 -- \
psql -U appuser -d winterarc
```

Inside psql, create a table and insert a test record:

```
CREATE TABLE lab_notes (id INT PRIMARY KEY, note TEXT);
INSERT INTO lab_notes VALUES (1, 'data-survives-pod-delete');
SELECT * FROM lab_notes;
\q
```

Now delete only the Pod and wait for its replacement:

```
kubectl get pvc -n day19-storage
kubectl delete pod pg-0 -n day19-storage
kubectl get pods -n day19-storage -w
```

After the replacement Pod is Ready, query the table again:

```
kubectl exec -n day19-storage pg-0 -- \
psql -U appuser -d winterarc -c \
"SELECT * FROM lab_notes;"
```

Success condition: the row **data-survives-pod-delete** still exists. Capture evidence of the Bound PVC, Pod recreation, and returned row.

8. Task F — inspect the storage chain

```
kubectl get statefulset,pods,pvc,pv -n day19-storage
kubectl describe pvc data-pg-0 -n day19-storage
kubectl get events -n day19-storage --sort-by=.lastTimestamp
```

In your README, explain: **StatefulSet Pod** → **PVC** → **PV** → **storage provisioner / node path.** Exact PV names and local paths vary by cluster.

9. Break & diagnose

Inspect PVC Events and write down at least two reasons a claim could stay Pending. Examples include a missing/mismatched StorageClass, provisioner issues, or unsatisfied storage requirements. Diagnose first; do not change settings blindly.

Quick troubleshooting

Symptom	Check
PVC Pending	Describe PVC; inspect Events and StorageClass/provisioner.
Pod not starting	Describe pg-0; check Events and whether the PVC is Bound.
Postgres restarts	Check logs and Secret key names.
Data seems missing	Verify the same PVC is attached and query the correct database/table.
PVC remains after Pod deletion	Expected: claims are normally retained when only the Pod is deleted.

10. Interview questions — explain the WHY

Try each answer out loud for 20–30 seconds before reading the suggested answer. Learn the reasoning, not just the definition.

Q1. PV vs PVC?
Suggested answer: A PV represents storage; a PVC is a workload's request for storage. Kubernetes binds a matching claim to a volume.

Q2. Why not store database files in the container filesystem?
Suggested answer: The writable layer is tied to the container lifecycle and is not the right place for durable database data. Persistent storage is designed to outlive a Pod.

Q3. What does a StorageClass do?
Suggested answer: It describes a class of storage and can trigger dynamic provisioning when a PVC requests storage.

Q4. What does volumeClaimTemplates do in a StatefulSet?
Suggested answer: It creates a dedicated PVC for each Pod identity, allowing the replacement Pod to reuse its claim.

Q5. Why did our data survive deleting pg-0?
Suggested answer: The StatefulSet recreated the Pod with the same identity and its persistent claim could reattach to the same volume.

Q6. Deployment vs StatefulSet?
Suggested answer: Deployment replicas are generally interchangeable. StatefulSets provide stable identities and ordered management, often paired with per-Pod persistent storage.

Q7. What does ReadWriteOnce mean?
Suggested answer: The volume can be mounted read-write by one node at a time. It does not simply mean only one Pod can ever use it; details depend on the storage driver.

Q8. Why use a Headless Service?
Suggested answer: It supports DNS identities for individual StatefulSet Pods rather than only one virtual service IP.

Q9. Why can a PVC stay Pending?
Suggested answer: The StorageClass may be missing or wrong, the provisioner may be unhealthy, or the requested storage may not be satisfiable. Inspect PVC Events first.

Q10. Does deleting a StatefulSet Pod delete its PVC?
Suggested answer: Normally, deleting only the Pod does not delete the claim created by volumeClaimTemplates. Verify retention settings and behavior before cleanup.

Q11. What is the risk of deleting a PVC?
Suggested answer: Depending on reclaim policy and storage setup, underlying storage/data may be deleted or retained. Treat PVC deletion as potentially destructive.

Q12. Is persistent storage the same as a backup?
Suggested answer: No. It helps with Pod/container failures but does not replace database backups, restore tests, or disaster recovery.

11. Mini challenge — prove it like an engineer

■ StorageClass identified. ■ PostgreSQL StatefulSet is Ready. ■ PVC is Bound. ■ Test row inserted. ■ Pod deleted and recreated. ■ Same row queried after recreation. ■ Storage chain explained. ■ Two Pending-PVC causes documented. ■ Interview answers written in your own words.

12. GitHub submission

Create **day19-storage-statefulsets/** containing **pg-headless.yaml**, **postgres-statefulset.yaml**, a README with commands and explanations, and screenshots of PVC Bound, Pod recreation, and the row before/after deletion. Never commit real credentials.

13. Cleanup — read before running

Save your evidence first. Deleting the namespace removes the lab objects; the fate of underlying data depends on reclaim policy and provisioner behavior. This is a disposable local lab only.

```
kubectl delete namespace day19-storage
kubectl get pv
```

Final goal: Demonstrate how Kubernetes connects a stateful Pod to durable storage—and prove that a database record survives Pod deletion.